Round-off errors and computer arithmetic

NUMERICAL ANALYSIS - JAN. 2026

MERCIALES, Mark Anthony C.
Course Instructor

Introduction

How is the arithmetic done in algebra or calculus different from arithmetic performed by a calculator or a computer?

https://roadie-metal.com/opiniao-mortes-aposentadorias-e-o-futuro-do-rock/

Introduction

Arithmetic in Algebra or Calculus

- *considers the set of real numbers*$(\mathbb{R})$
- *have precise statements like*

$$3 + 3 = 6$$

$$(3)(3) = 9$$

$$\left(\sqrt{3}\right)^2 = 3$$

Introduction

Arithmetic in computers or calculators

- *considers a subset of real numbers*$(\mathbb{R})$
- *have precise statements like*

$$3 + 3 = 6$$

$$(3)(3) = 9$$

but not

$$\left(\sqrt{3}\right)^2 = 3$$

Introduction

In our traditional mathematical world(over $\mathbb{R}$), we permit numbers with an <u>infinite</u> number of digits.

- ✓ *we have rational or irrational numbers*

  -*e.g.* $1/3$ which is a rational number which has a non-terminating decimal expansion given by 0.3333333….

Introduction

In our traditional mathematical world(over $\mathbb{R}$), we permit numbers with an <u>infinite</u> number of digits.

- ✓ *we have rational or irrational numbers*

  -*e.g.* $\sqrt{3}$ which is defined as the unique positive number that when multiplied by itself produces the integer 3.

  - $\sqrt{3}$ has a non-terminating, non-repeating decimal expansion.

Introduction

In the computational world, each representable number has only a fixed and finite number of digits.

- ✓ *only rational numbers (not even all rational numbers)* can be represented exactly

  -*e.g.* $1/3$ is a rational number whose decimal representation has infinitely many repeating digits.

Introduction

In the computational world, each representable number has only a fixed and finite number of digits.

- ✓ *only rational numbers (not even all rational numbers)* can be represented exactly

However, we note that

- ✓ *all numbers are given an approximate representation, which is acceptable in most situations.*

Introduction

Sometimes, computer arithmetic results to some problems because of discrepancy between the approximate representation and the actual number.

- ✓ **Round-off error:** The error that is produced when a calculator or computer is used to perform real number calculations.

Introduction

✓ Round-off error

Suppose we try to add **0.1 + 0.2** in most programming languages:

python Copy code

```python
>>> 0.1 + 0.2
0.30000000000000004
```

What happened?

- 0.1 and 0.2 **cannot be exactly represented** in binary.
- So, when the computer adds them, the result isn't exactly 0.3, but something extremely close: `0.30000000000000004`.
- This tiny difference is a **round-off error**.

Binary Machine Numbers

- In 1985, the Institute for Electrical and Electronic Engineers(IEEE) published a report called Binary Floating Point Arithmetic Standard 754–1985. An updated version was published in 2008 as IEEE 754-2008.

Binary Machine Numbers

- Binary Floating Point Arithmetic Standard
  - ✓ provides standards for binary and decimal floating point numbers, formats for data interchange, algorithms for rounding arithmetic operations, and for the handling of exceptions.
  - ✓ generally followed by all microcomputer manufacturers using floating-point hardware.

Binary Machine Numbers

- Representation of Real Numbers
  - ✓ A 64-bit (binary digit) representation is used for a real number

    -The first bit is a sign indicator, denoted by s.

    -This is followed by an 11-bit exponent, c, called the <u>characteristic</u>, and a 52-bit binary fraction, f, called the <u>mantissa</u>.

    -The base for the exponent is 2

Binary Machine Numbers

- Representation of Real Numbers
  - ✓ Since 52 binary digits correspond to between 16 and 17 decimal digits, we can assume that a number represented in this system has at least 16 decimal digits of precision.

Binary Machine Numbers

❖ Representation of Real Numbers

- ✓ The exponent of 11 binary digits gives a range of 0 to $2^{11} - 1 = 2047$.
- ✓ To ensure that numbers with small magnitude are equally representable, 1023 is subtracted from the characteristic, so the range of the exponent is actually from -1023 to 1024.

Binary Machine Numbers

❖ Representation of Real Numbers

✓ To save storage and provide a unique representation for each floating-point number, a normalization is imposed. Using this system gives a floating-point number of the form

e

$$(-1)^s 2^{\boxed{c-1023}}(1+f)$$

$$c = a_1 2^{10} + a_2 2^9 + a_3 2^8 + \cdots + a_{10} 2^1 + a_{11} 2^0$$

$$f = b_1 2^{-1} + b_2 2^{-2} + b_3 2^{-3} + \cdots + b_{51} 2^{-51} + b_{52} 2^{-52}$$

Binary Machine Numbers

❖ Representation of Real Numbers

Ex. Consider the machine number

0 10000000011 1011100100010000000000000000000000000000000000000000.

0 10000000011 1011100100010000000000000000000000000000000000000000.

The leftmost bit is $s = 0$, which indicates that the number is positive. The next 11 bits, 10000000011, give the characteristic and are equivalent to the decimal number

$$c = 1 \cdot 2^{10} + 0 \cdot 2^{9} + \cdots + 0 \cdot 2^{2} + 1 \cdot 2^{1} + 1 \cdot 2^{0} = 1024 + 2 + 1 = 1027.$$

Binary Machine Numbers

❖ Representation of Real Numbers

Ex. Consider the machine number

0 10000000011 1011100100010000000000000000000000000000000000000000.

$a_1 \longrightarrow a_{11}, b_1 \longrightarrow b_{52}$

0 10000000011 1011100100010000000000000000000000000000000000000000.

$a_1 = 1$

$a_2 = a_3 = \cdots = a_9 = 0$

$c = (1)2^{10} + \cancel{(0)2^9} + \cancel{(0)2^8} + \cancel{(0)2^7} + \cancel{(0)2^6} + \cancel{(0)2^5} + \cancel{(0)2^4} + \cancel{(0)2^3} + \cancel{(0)2^2} + (1)2^1 + (1)2^0$

$a_1 \quad a_2 \quad a_3 \quad a_4 \quad a_5 \quad a_6 \quad a_7 \quad a_8 \quad a_9 \quad a_{10} \quad a_{11}$

$a_{10} = 1$

$c = (1)2^{10} + (1)(2)^1 + (1)2^0$

$a_{11} = 1$

The leftmost bit is $s = 0$, which indicates that the number is positive. The next 11 bits, 10000000011, give the characteristic and are equivalent to the decimal number

$$c = 1 \cdot 2^{10} + 0 \cdot 2^9 + \cdots + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 1024 + 2 + 1 = 1027.$$

Binary Machine Numbers

0 10000000011 1011100100010000000000000000000000000000000000000000.

b_1 $\qquad b_{13} = b_{14} = \cdots = b_{52} = 0$

$f = (1)2^{-1} + (0)2^{-2} + (1)2^{-3} + (1)2^{-4} + (1)2^{-5} + (0)2^{-6} + (0)2^{-7} + (1)2^{-8} + (0)2^{-9} + (0)2^{-10} + (0)2^{-11} + (1)2^{-12}$

The exponential part of the number is, therefore, $2^{1027-1023} = 2^4$. The final 52 bits specify that the mantissa is

$$f = 1 \cdot \left(\frac{1}{2}\right)^1 + 1 \cdot \left(\frac{1}{2}\right)^3 + 1 \cdot \left(\frac{1}{2}\right)^4 + 1 \cdot \left(\frac{1}{2}\right)^5 + 1 \cdot \left(\frac{1}{2}\right)^8 + 1 \cdot \left(\frac{1}{2}\right)^{12}.$$

$+ (0)2^{-13} + \cdots + (0)2^{-52}$

$f = (1)2^{-1} + (1)2^{-3} + (1)2^{-4} + (1)2^{-5} + (1)2^{-8} + (1)2^{-12}$

$= \frac{1}{2} + \frac{1}{2^3} + \frac{1}{2^4} + \frac{1}{2^5} + \frac{1}{2^8} + \frac{1}{2^{12}}$ ✓

$= \frac{1}{2} + \left(\frac{1}{2}\right)^3 + \left(\frac{1}{2}\right)^4 + \left(\frac{1}{2}\right)^5 + \left(\frac{1}{2}\right)^8 + \left(\frac{1}{2}\right)^{12}$ ✓

Binary Machine Numbers

0 10000000011 ~~1011100100~~1000000000000000000000000000000000000000.

b_1 $\qquad b_{13} = b_{14} = \cdots = b_{52} = 0$

$$f = (1)2^{-1} + \cancel{(0)2^{-2}} + (1)2^{-3} + (1)2^{-4} + (1)2^{-5} + \cancel{(0)2^{-6}} + \cancel{(0)2^{-7}} + (1)2^{-8} + \cancel{(0)2^{-9}} + \cancel{(0)2^{-10}} + \cancel{(0)2^{-11}} + (1)2^{-12}$$

$$\cancel{+ (0)2^{-13} + \cdots + (0)2^{-52}}$$

b_{13} $\qquad b_{52}$

$$f = (1)2^{-1} + (1)2^{-3} + (1)2^{-4} + (1)2^{-5} + (1)2^{-8} + (1)2^{-12}$$

$$= \frac{1}{2} + \frac{1}{2^3} + \frac{1}{2^4} + \frac{1}{2^5} + \frac{1}{2^8} + \frac{1}{2^{12}} \quad \checkmark$$

$$= \frac{1}{2} + \left(\frac{1}{2}\right)^3 + \left(\frac{1}{2}\right)^4 + \left(\frac{1}{2}\right)^5 + \left(\frac{1}{2}\right)^8 + \left(\frac{1}{2}\right)^{12} \quad \checkmark$$

Binary Machine Numbers

$$c = 1 \cdot 2^{10} + 0 \cdot 2^9 + \cdots + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 1024 + 2 + 1 = 1027.$$

$$f = 1 \cdot \left(\frac{1}{2}\right)^1 + 1 \cdot \left(\frac{1}{2}\right)^3 + 1 \cdot \left(\frac{1}{2}\right)^4 + 1 \cdot \left(\frac{1}{2}\right)^5 + 1 \cdot \left(\frac{1}{2}\right)^8 + 1 \cdot \left(\frac{1}{2}\right)^{12}.$$

As a consequence, this machine number precisely represents the decimal number

$$(-1)^s 2^{c-1023}(1+f) = (-1)^0 \cdot 2^{\overset{4}{1027-1023}} \left(1 + \left(\frac{1}{2} + \frac{1}{8} + \frac{1}{16} + \frac{1}{32} + \frac{1}{256} + \frac{1}{4096}\right)\right)$$

$$= 27.56640625.$$

Binary Machine Numbers

However, the next smallest machine number is

$$0\ 10000000011\ 1011100100001111111111111111111111111111111111111111,$$

and the next largest machine number is

$$0\ 10000000011\ 1011100100010000000000000000000000000000000000000001.$$

NOTE:

- ✓ <u>next smallest</u> means the largest representable number that is *strictly less than* the given number
- ✓ <u>next largest</u> means the smallest representable number that is *strictly greater than* the given number

Binary Machine Numbers

This means that our original machine number represents not only 27.56640625, but also

- ✓ half of the real numbers that are between 27.56640625 and the next smallest machine number,
- ✓ as well as half the numbers between 27.56640625 and the next largest machine number.

Binary Machine Numbers

- ✓ To be precise, it represents any real number in the interval

$$[27.56640624999999822364316059974953532218933105468 75,$$
$$27.56640625000000177635683940025046467781066894531 25).$$

How to obtain the half-open interval?

Binary Machine Numbers

Unit in the Last Place(ULP)

- ✓ the **distance between two consecutive machine numbers** (the smallest difference you can represent at that magnitude).
- ✓ the smallest amount by which you can increase a floating-point number without it being rounded back to the same value

Binary Machine Numbers

Unit in the Last Place(ULP)

- ✓ For **normalized number**($\boldsymbol{x}$) in double precision(64 bits)

$$x = (-1)^s 2^{c-1023}(1+f)$$

we have

$$ULP(x) = 2^{e-p}$$

wherein

$$e = c - 1023$$

$$p = 53$$

Binary Machine Numbers

Unit in the Last Place(ULP)

- ✓ For **normalized number**($\boldsymbol{x}$) in single precision(32bits)

$$x = (-1)^s 2^{c-127}(1+f)$$

we have

$$ULP(x) = 2^{e-p}$$

wherein

$$e = c - 127$$

$$p = 24$$

Binary Machine Numbers

- A machine number with floating-point number (x) represents all real numbers in the interval

$$\left[\frac{x_- + x}{2}, \frac{x + x_+}{2}\right)$$

wherein

x_- is the largest machine number smaller

x_+ is the smallest machine number larger

$$x_- = x - 2^{e-p}$$
$$x_+ = x + 2^{e-p}$$

Binary Machine Numbers

- A machine number with floating-point number (x) represents all real numbers in the interval

$$\left[\frac{x_- + x}{2}, \frac{x + x_+}{2}\right) \Longrightarrow \left[x - \frac{1}{2}ULP, x + \frac{1}{2}ULP\right)$$

wherein

x_- is the largest machine number smaller

x_+ is the smallest machine number larger

$$x_- = x - ULP$$
$$x_+ = x + ULP$$

Binary Machine Numbers

❖ Smallest and largest normalized machine number

The smallest normalized positive number that can be represented has $s = 0$, $c = 1$, and $f = 0$ and is equivalent to

$$2^{-1022} \cdot (1 + 0) \approx 0.22251 \times 10^{-307},$$

and the largest has $s = 0$, $c = 2046$, and $f = 1 - 2^{-52}$ and is equivalent to

$$2^{1023} \cdot (2 - 2^{-52}) \approx 0.17977 \times 10^{309}.$$

Binary Machine Numbers

❖ Smallest and largest r

The smallest normalized p 1,
and $f = 0$ and is equivalent to

Double precision: **11 exponent bits**

Exponent range for **normalized numbers**: $1 \leq c \leq 2046$

- `c = 0` → subnormal
- `c = 2047` → infinity/NaN

So **smallest positive normalized number** → use **c = 1**

Largest positive number → **largest exponent for normalized number**: $c = 2046$

and the largest has $s = 0$, $c = 2046$, and $f = 1 - 2^{-52}$ and is equivalent to

$$2^{1023} \cdot (2 - 2^{-52}) \approx 0.17977 \times 10^{309}.$$

To maximize the value, set **all fraction bits = 1**

Fraction = $f = 1 - 2^{-52}$ (all 52 stored fraction bits 1)

Why not exactly 1? Because the fraction is stored in **52 bits**, so max fraction < 1:

$$f = \frac{2^{52} - 1}{2^{52}} = 1 - 2^{-52}$$

Binary Machine Numbers

❖ Smallest and largest normalized machine number

Numbers occurring in calculations that have a magnitude less than

$$2^{-1022} \cdot (1 + 0)$$

size of the number(x) ignoring its sign($|x|$).

result in **underflow** and are generally set to zero. Numbers greater than

$$2^{1023} \cdot (2 - 2^{-52})$$

result in **overflow** and typically cause the computations to stop (unless the program has been designed to detect this occurrence). Note that there are two representations for the number zero; a positive 0 when $s = 0$, $c = 0$ and $f = 0$, and a negative 0 when $s = 1$, $c = 0$ and $f = 0$.

Decimal Machine Numbers

- The use of binary digits tends to conceal the computational difficulties that occur when a finite collection of machine numbers is used to represent all the real numbers.
  - ✓ we will use more familiar decimal numbers instead of binary representation.
  - ✓ normalized decimal floating-point form:

$$\pm 0.d_1 d_2 \ldots d_k \times 10^n$$

wherein $1 \leq d_1 \leq 9$ and $0 \leq d_i \leq 9$ for each $i = 2, 3, \ldots, k$.

Decimal Machine Numbers

$$\pm 0.d_1 d_2 \ldots d_k \times 10^n$$

wherein $1 \leq d_1 \leq 9$ and $0 \leq d_2 \leq 9$ for each $i = 2, 3, \ldots, k$.

- ✓ numbers of
  numbers.

Moving the decimal point:

Right by a places → decrease exponent by a

Left by a places → increase exponent by a

Ex. 0.297211

while these two are not 7-digit nor 9-digit machine numbers:

2.9721189×10^6 $\qquad$ 0.029721189×10^8

Decimal Machine Numbers

- ✓ Any positive real number within the numerical range of the machine can be normalized to the form

$$y = 0.d_1 d_2 \dots d_k d_{k+1} d_{k+2} \dots \times 10^n$$

- ✓ The floating-point form of y, denoted $fl(y)$, is obtained by terminating the mantissa of y at k decimal digits.
- ✓ There are two common ways of performing this termination: <u>chopping</u> and <u>rounding.</u>

Decimal Machine Numbers

$$y = 0.d_1 d_2 \ldots d_k d_{k+1} d_{k+2} \ldots \times 10^n$$

❖ <u>Chopping</u>

✓ This produces the floating-point form

$$fl(y) = 0.d_1 d_2 \ldots d_k \times 10^n$$

Example: Determine the five-digit chopping values of the irrational number π.

Decimal Machine Numbers

Example: Determine the five-digit chopping values of the irrational number π.

Solution: The number π has an infinite decimal expansion of the form $\pi = 3.14159265\ldots$.

In normalized decimal form, we have

$$\pi = 0.314159265\ldots \times 10^1$$

The floating-point form of π using five-digit chopping is

$$fl(\pi) = 0.31415 \times 10^1 = 3.1415$$

Decimal Machine Numbers

$$y = 0.d_1 d_2 \ldots d_k d_{k+1} d_{k+2} \ldots \times 10^n$$

❖ <u>Rounding</u>

✓ adds $5 \times 10^{n-(k+1)}$ to y and then chops the result to obtain a number of the form

$$fl(y) = 0.\delta_1 \delta_2 \ldots \delta_k \times 10^n$$

Example: Determine the five-digit rounding values of the irrational number π.

Decimal Machine Numbers

Example: Determine the five-digit rounding values of the irrational number π.

Solution: $\pi = 3.14159265 \ldots$

normalized decimal form:

$$\pi = 0.314159265 \ldots \times 10^1$$

We have $k = 5$:

$$\begin{aligned} 5 \times 10^{1-(5+1)} &= 5 \times 10^{1-(5+1)} \\ &= 5 \times 10^{1-6} \\ &= 0.000005 \times 10^1 \end{aligned}$$

Decimal Machine Numbers

Example: Determine the five-digit rounding values of the irrational number π.

Solution: $\pi = 0.314159265\ldots \times 10^1$

$$5 \times 10^{1-(5+1)} = 0.000005 \times 10^1$$

Hence,

$$\pi + (5 \times 10^{1-(5+1)}) = 0.314159265\ldots \times 10^1$$
$$+0.000005 \times 10^1$$
$$= 0.314164265\ldots \times 10^1$$

$$fl(y) = 0.31416 \times 10^1$$
$$= 3.1416$$

Decimal Machine Numbers

Absolute Error

- ✓ Suppose that p^* is an approximation to p. The absolute error is given by

$$E_a = |p - p^*|$$

Example: Determine the absolute error when approximating p by p^* when

$$p = 0.3000 \times 10^1$$

and $p^* = 0.3100 \times 10^1$

Decimal Machine Numbers

Absolute Error

- ✓ Suppose that p^* is an approximation to p. The absolute error is given by

$$E_a = |p - p^*|$$

Example: Determine the absolute error when approximating p by p^* when

$$p = 0.3000 \times 10^{-3}$$

and $p^* = 0.3100 \times 10^{-3}$

Decimal Machine Numbers

Absolute Error

- ✓ Suppose that p^* is an approximation to p. The absolute error is given by

$$E_a = |p - p^*|$$

Example: Determine the absolute error when approximating p by p^* when

$$p = 0.3000 \times 10^{-4}$$

and $p^* = 0.3100 \times 10^{-4}$

Decimal Machine Numbers

Relative Error

- ✓ Suppose that p^* is an approximation to p. The absolute error is given by

$$E_r = \frac{|p - p^*|}{|p|}$$

provided that $p \neq 0$.

Decimal Machine Numbers

Relative Error

$$E_r = \frac{|p - p^*|}{|p|}$$

Example: Determine the relative error when approximating p by p^* when

$$p = 0.3000 \times 10^1$$

and $p^* = 0.3100 \times 10^1$

Decimal Machine Numbers

Relative Error

$$E_r = \frac{|p - p^*|}{|p|}$$

Example: Determine the relative error when approximating p by p^* when

$$p = 0.3000 \times 10^{-3}$$

and $p^* = 0.3100 \times 10^{-3}$

Decimal Machine Numbers

Relative Error

$$E_r = \frac{|p - p^*|}{|p|}$$

Example: Determine the relative error when approximating p by p^* when

$$p = 0.3000 \times 10^4$$

and $p^* = 0.3100 \times 10^4$

Decimal Machine Numbers

Absolute and Relative Error

Example: Determine the absolute and relative errors when approximating p by p^* when

1. $p = 0.3000\times10^{1}$ and $p^* = 0.3100\times10^{1}$
2. $p = 0.3000\times10^{-3}$ and $p^* = 0.3100\times10^{-3}$
3. $p = 0.3000\times10^{4}$ and $p^* = 0.3100\times10^{4}$

Answers:

1. Absolute Error: 0.01; Relative Error: $0.333\overline{3}\times10^{-1}$
2. Absolute Error: 0.01×10^{-3}; Relative Error: $0.333\overline{3}\times10^{-1}$
3. Absolute Error: 0.01×10^{4}; Relative Error: $0.333\overline{3}\times10^{-1}$

Decimal Machine Numbers

Absolute and Relative Error

- ✓ The relative error is generally a better measure of accuracy than the absolute error because it takes into consideration the size of the number being approximated.
- ✓ The relative error E_r suggests that the true value p lies within $p \pm (E_r|p|)$.

Decimal Machine Numbers

Absolute and Relative Error

Compute the exact product of 3.6894 and 5.1549, then round the result to two decimal digits. Determine the absolute (true) error and the relative error of the rounded result.

Decimal Machine Numbers

Absolute and Relative Error

Determine the absolute (true) error and the relative error when the area of a circle with radius $r = 3.689$ is computed using two decimal digits.

Decimal Machine Numbers

Significant digits

- ✓ The number p^* is said to approximate p to t significant digits (or figures) if t is the largest nonnegative integer for which

$$\frac{|p - p^*|}{|p|} \leq 5 \times 10^{-t}$$

Decimal Machine Numbers

Significant digits

Example:

Let the true value be: $p = 3.14159$. Suppose we approximate p by $p^* = 3.14$.

$$\frac{|p - p^*|}{|p|} \Longrightarrow \frac{|3.14159 - 3.14|}{|3.14159|} \approx 5.06\times10^{-4} \leq 5\times10^{-3}$$

$$> 5\times10^{-4}$$

Hence, we have "$p^* = 3.14$ approximates $p = 3.14159$ to 3 significant digits."

Decimal Machine Numbers

Significant digits

Example:

Let the true value be: $p = 1.9999$. Suppose we approximate p by $p^* = 1.90$.

$$\frac{|p - p^*|}{|p|} \Longrightarrow \frac{|1.9999 - 1.90|}{|1.999|} \approx 0.04995 \quad \begin{array}{l} \leq 5\times10^{-2} \\ > 5\times10^{-3} \end{array}$$

Hence, we have "$p^* = 1.90$ approximates $p = 1.9999$ to 2 significant digits."

Decimal Machine Numbers

Significant digits

Example:

Let the true value be: $p = 1.9999$. Suppose we approximate p by $p^* = 1.90$.

$$\frac{|p - p^*|}{|p|} \Longrightarrow \frac{|1.9999 - 1.90|}{|1.999|} \approx 0.04995 \quad \begin{array}{l} \leq 5\times10^{-2} \\ > 5\times10^{-3} \end{array}$$

Hence, we have "$p^* = 1.90$ approximates $p = 1.9999$ to 2 significant digits."

Decimal Machine Numbers

Significant digits

Example:

Let the true value be: $p = 1.9999$. Suppose we approximate p by $p^* = 1.90$.

$$\frac{|p - p^*|}{|p|} \Longrightarrow \frac{|1.9999 - 1.90|}{|1.999|} \approx 0.04995 \quad \begin{array}{l} \leq 5\times10^{-2} \\ > 5\times10^{-3} \end{array}$$

Hence, we have "$p^* = 1.90$ approximates $p = 1.9999$ to 2 significant digits."

Decimal Machine Numbers

Maximum absolute error

$$\frac{|p - p^*|}{|p|} \leq 5\times10^{-t}$$

$$\Leftrightarrow \quad |p - p^*| \leq (5\times10^{-t})|p|$$

✓ the **maximum absolute error** depends on both the **number of significant digits** t and the magnitude of p.

$$\max|p - p^*| = (5\times10^{-t})|p|$$

Decimal Machine Numbers

Maximum absolute error

$$max|p - p^*| = (5\times10^{-t})|p|$$

Example: Determine the maximum absolute error given $p = 0.1$ when p^* agrees with p to four significant digits.

$$\max|p - p^*| = (5\times10^{-4})|0.1|$$

$$= 0.00005 \quad Ans.$$

p	0.1	0.5	100	1000	5000	9990	10000
$\max\|p - p^*\|$	0.00005	0.00025	0.05	0.5	2.5	4.995	5.

Decimal Machine Numbers

Maximum relative error of floating point form[chopping]

Let $y = 0.d_1 d_2 \dots d_k d_{k+1} d_{k+2} \dots \times 10^n$. Given $p = y$ and $p^* = fl(y)$ done by k chopping,

$$\frac{|p - p^*|}{|p|} \Longrightarrow \frac{|y - fl(y)|}{|y|}$$

$$= \frac{|0.d_1 d_2 \dots d_k d_{k+1} d_{k+2} \dots \times 10^n - 0.d_1 d_2 \dots d_k \times 10^n|}{|0.d_1 d_2 \dots d_k d_{k+1} d_{k+2} \dots \times 10^n|}$$

Decimal Machine Numbers

Maximum relative error of floating point form[chopping]

$$= \frac{|0.d_1 d_2 \dots d_k d_{k+1} d_{k+2} \dots \times 10^n - 0.d_1 d_2 \dots d_k \times 10^n|}{|0.d_1 d_2 \dots d_k d_{k+1} d_{k+2} \dots \times 10^n|}$$

$$= \frac{|0.0_1 0_2 \dots 0_k d_{k+1} d_{k+2} \dots \times 10^n|}{|0.d_1 d_2 \dots d_k d_{k+1} d_{k+2} \dots \times 10^n|} = \frac{\left|0.d_{k+1} d_{k+2} \dots \times 10^{n-k}\right|}{|0.d_1 d_2 \dots d_k d_{k+1} d_{k+2} \dots \times 10^n|}$$

$$= \frac{|0.d_{k+1} d_{k+2} \dots \times 10^n|}{|0.d_1 d_2 \dots d_k d_{k+1} d_{k+2} \dots \times 10^n|} \times 10^{-k}$$

Decimal Machine Numbers

Maximum relative error of floating point form[chopping]

$$\frac{|y - fl(y)|}{|y|} = \frac{|0.d_{k+1}d_{k+2} \dots \times 10^n|}{|0.d_1 d_2 \dots d_k d_{k+1} d_{k+2} \dots \times 10^n|} \times 10^{-k}$$

Hence,

$$\frac{|y - fl(y)|}{|y|} \leq \frac{1}{0.1} \times 10^{-k} = 10^{-k+1}$$

since $d_1 \neq 0$, the minimal value of the denominator is 0.1 and the numerator is bounded above by 1.

Decimal Machine Numbers

Maximum relative error of floating point form[rounding]

We can similarly show that a bound for the relative error when using k-digit rounding arithmetic is given by

$$\frac{|y - fl(y)|}{|y|} = 0.5 \times 10^{-k+1}$$

Binary Numbers(Base 2) and Decimal Numbers(Base 10)

Binary to decimal

If we have a binary number of the form

$$a_n a_{n-1} a_{n-2} \dots a_1 a_0 . a_{-1} a_{-2} a_{-3} \dots$$

Then its decimal value is:

$$a_n 2^n + a_{n-1} 2^{n-1} + \cdots + a_1 2^1 + a_0 2^0$$
$$+ a_{-1} 2^{-1} + a_{-2} 2^{-2} + a_{-3} 2^{-3} \dots$$

where each a_i is 1 or 0.

Binary Numbers(Base 2) and Decimal Numbers(Base 10)

Example: Convert 1011.01_2 to decimal

Step 1: Split integer and fractional parts:

- Integer part: 1011_2
- Fractional part: 0.01_2

Integer part:

$$1011_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 8 + 0 + 2 + 1 = 11$$

Fractional part:

$$0.01_2 = 0 \cdot 2^{-1} + 1 \cdot 2^{-2} = 0 + 0.25 = 0.25$$

Combine both:

$$1011.01_2 = 11 + 0.25 = 11.25_{10}$$

Binary Numbers(Base 2) and Decimal Numbers(Base 10)

Decimal to binary

- Converting the integer part to binary

✓ Take integer part of the given number.

✓ Divide it by 2. Write down the **remainder** (0 or 1).

✓ Use the **quotient** for the next division.

✓ Repeat until the quotient is 0.

✓ The **binary number** is the remainders written **from bottom to top**.

Binary Numbers(Base 2) and Decimal Numbers(Base 10)

Decimal to binary

- Converting the fraction/decimal part to binary

✓ Take the fraction part/decimal part of the given number.

✓ Multiply by 2.

✓ The **integer part** of the result (0 or 1) becomes the next binary digit.

✓ Keep the **fraction part** and repeat.

✓ Stop when the fraction becomes 0 or after desired precision.

Binary Numbers(Base 2) and Decimal Numbers(Base 10)

Example: Convert 13 *to binary.*

1. $13 \div 2 = 6$ remainder **1**
2. $6 \div 2 = 3$ remainder **0**
3. $3 \div 2 = 1$ remainder **1**
4. $1 \div 2 = 0$ remainder **1**

$13_{10} = 1101_2$

Binary Numbers(Base 2) and Decimal Numbers(Base 10)

Example: Convert 0.0625 *to binary.*

1. $0.0625 \times 2 = 0.125$ → integer part = **0**
2. $0.125 \times 2 = 0.25$ → integer part = **0**
3. $0.25 \times 2 = 0.5$ → integer part = **0**
4. $0.5 \times 2 = 1.0$ → integer part = **1**

$$0.0625_{10} = 0001_2$$

Binary Numbers(Base 2) and Decimal Numbers(Base 10)

Example: Convert 13.0625 *to binary.*

$$13_{10} = 1101_2$$

$$0.0625_{10} = 0001_2$$

$$13.0625_{10} = 1101.0001_2$$

Binary Numbers(Base 2) and Decimal Numbers(Base 10)

Example: Convert 13.0625 *to* IEEE-754 double-precision (64-bit) floating-point number

$$13.0625_{10} = 1101.0001_2$$

$$\begin{aligned}
1101.0001_2 &= (1)2^3 + (1)2^2 + (0)2^1 + (1)2^0 \\
&\quad +(0)2^{-1} + (0)2^{-2} + (0)2^{-3} + (0)2^{-4} \\
&= 2^3 + 2^2 + 2^0 + 2^{-4} \\
&= 2^3(1 + 2^{-1} + 2^{-3} + 2^{-7}) \\
&= 2^3(1.1010001_2)
\end{aligned}$$

normalized floating point form

Binary Numbers(Base 2) and Decimal Numbers(Base 10)

Example: Convert 13.0625 *to* IEEE-754 double-precision (64-bit) floating-point number

$2^3(1.1010001_2)$ normalized floating point form

$$c = e + 1023$$
$$= 3 + 1023$$
$$= 1026_{10}$$
$$= 1026$$
$$= 10000000010_2$$

Binary Numbers(Base 2) and Decimal Numbers(Base 10)

Step	Dividend	÷ 2	Quotient	Remainder
1	1026	÷2	513	0
2	513	÷2	256	1
3	256	÷2	128	0
4	128	÷2	64	0
5	64	÷2	32	0
6	32	÷2	16	0
7	16	÷2	8	0
8	8	÷2	4	0
9	4	÷2	2	0
10	2	÷2	1	0
11	1	÷2	0	1

Binary Numbers(Base 2) and Decimal Numbers(Base 10)

Example: Convert 13.0625 *to* IEEE-754 double-precision (64-bit) floating-point number

$2^3(1.1010001_2)$ normalized floating point form

$e = 10000000010_2$

$s = 0$

$$0\ 10000000010\ 1010001\underbrace{00\ldots0}_{b_8 b_9 \ldots b_{52}}$$